Cloudflare Solutions Engineer – Technical Project

Pre-requisite:

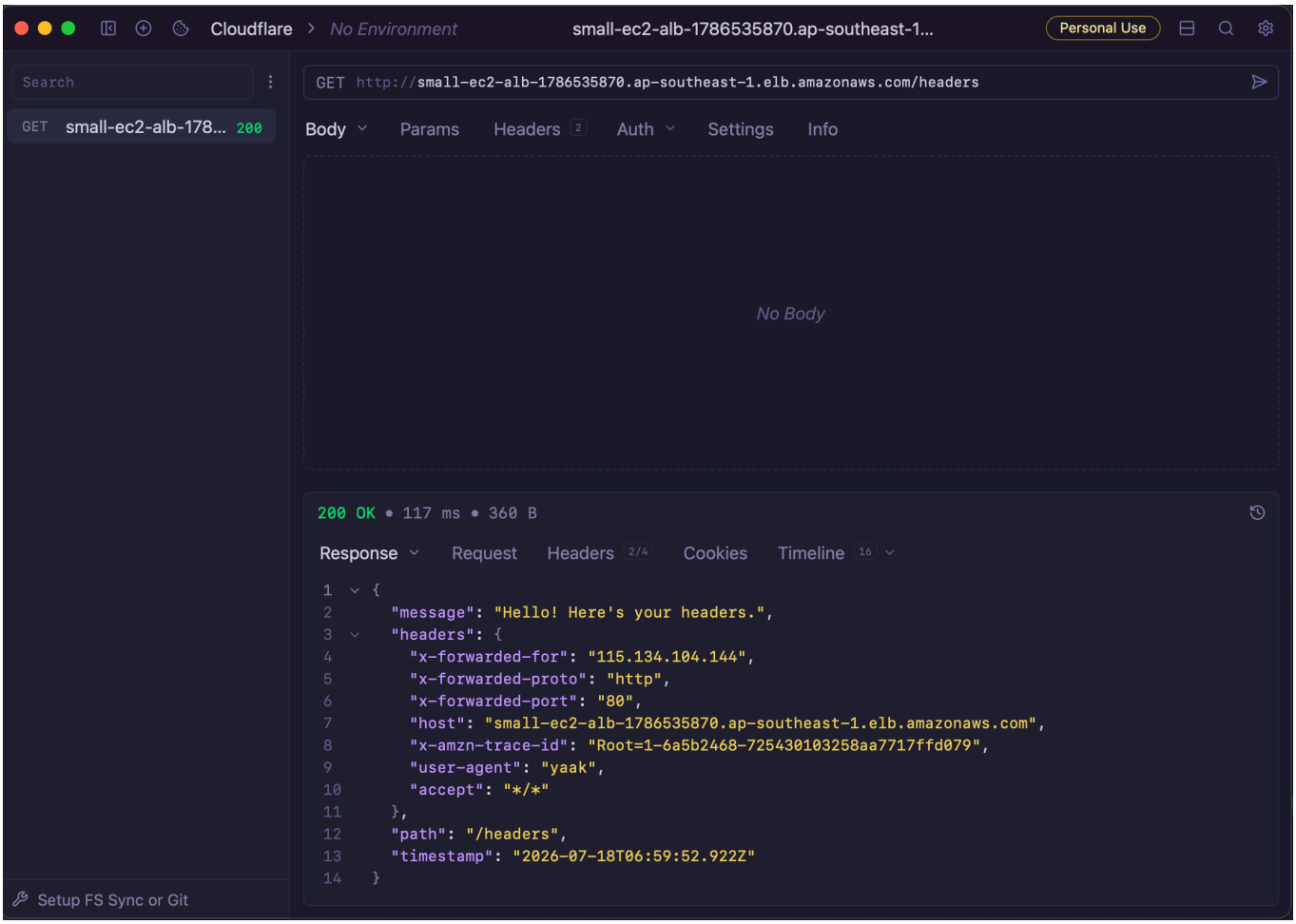
> 1. Please add a domain (new or existing, e.g. yourwebsite.com) to Cloudflare. Cloudflare Free plan is sufficient.
> 2. Activate on Cloudflare by following the steps to change the nameservers at your DNS Registrar.

Bought a domain on Cloudflare "wilson-here.uk".

Step 1

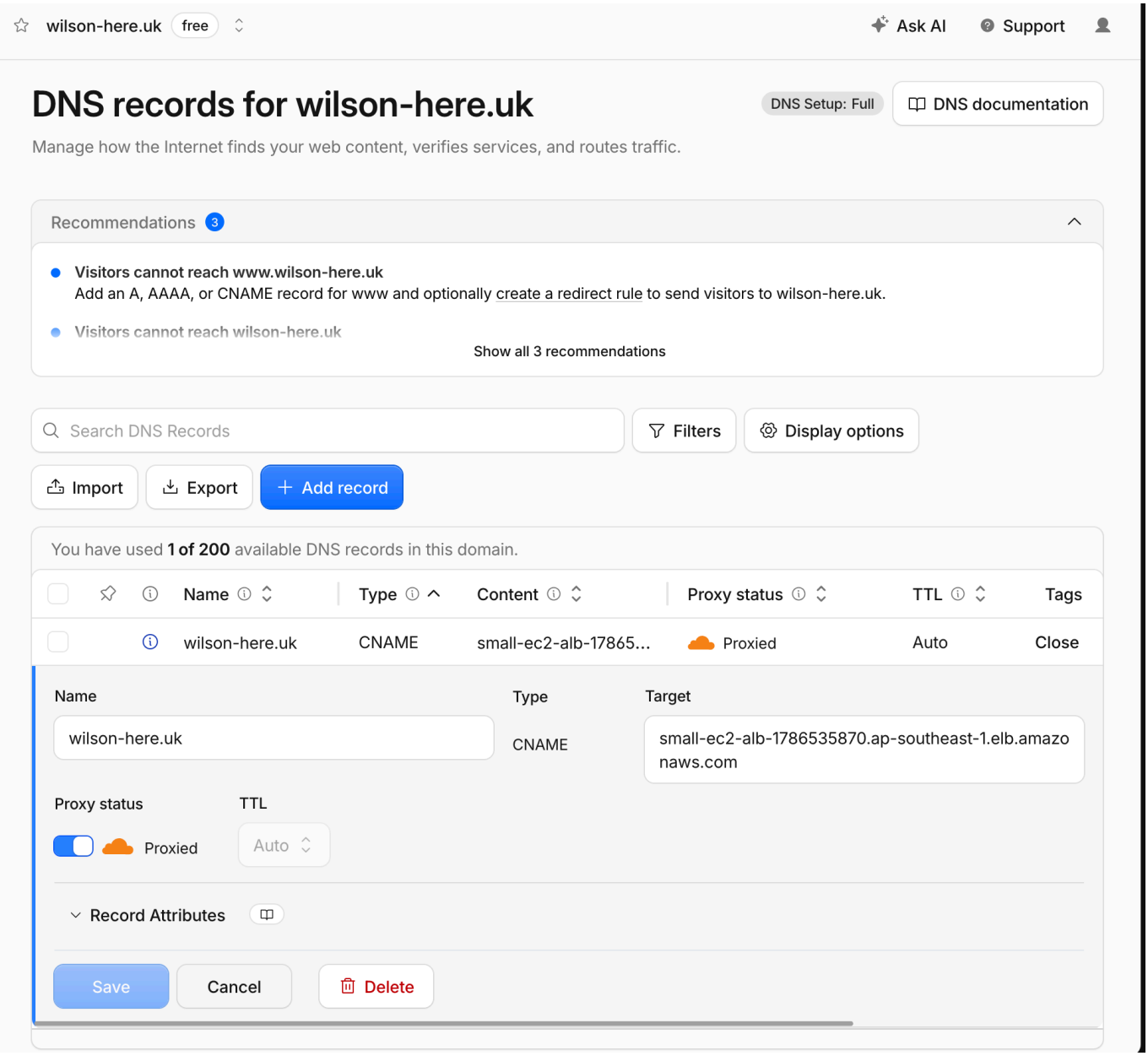
> Create an origin web server on a platform of your choosing. This could be in AWS, Google Cloud, DigitalOcean, your Raspberry Pi, etc.
>
> This web server must run an endpoint that returns all HTTP request headers in the body of the HTTP response.
>
> The web server can be something that you have written yourself (e.g. in JavaScript, Python, etc) or by using a 3rd party application.

- Deployed an EC2 using Terraform.
- Ran a NodeJS http server.



Step 2

> Proxy traffic to this server through Cloudflare. Add necessary configurations on Cloudflare.



Default TLS mode was Full. However HTTPS connection won't work due to no certificate set up on ALB.

Step 3

> Secure the communication between Cloudflare and your Origin Server with a non-Cloudflare provisioned TLS certificate using at least the Full-Strict mode on Cloudflare

Public cert was setup with wildcard (*.wilson-here.uk) using ACM on AWS and attached to ALB.

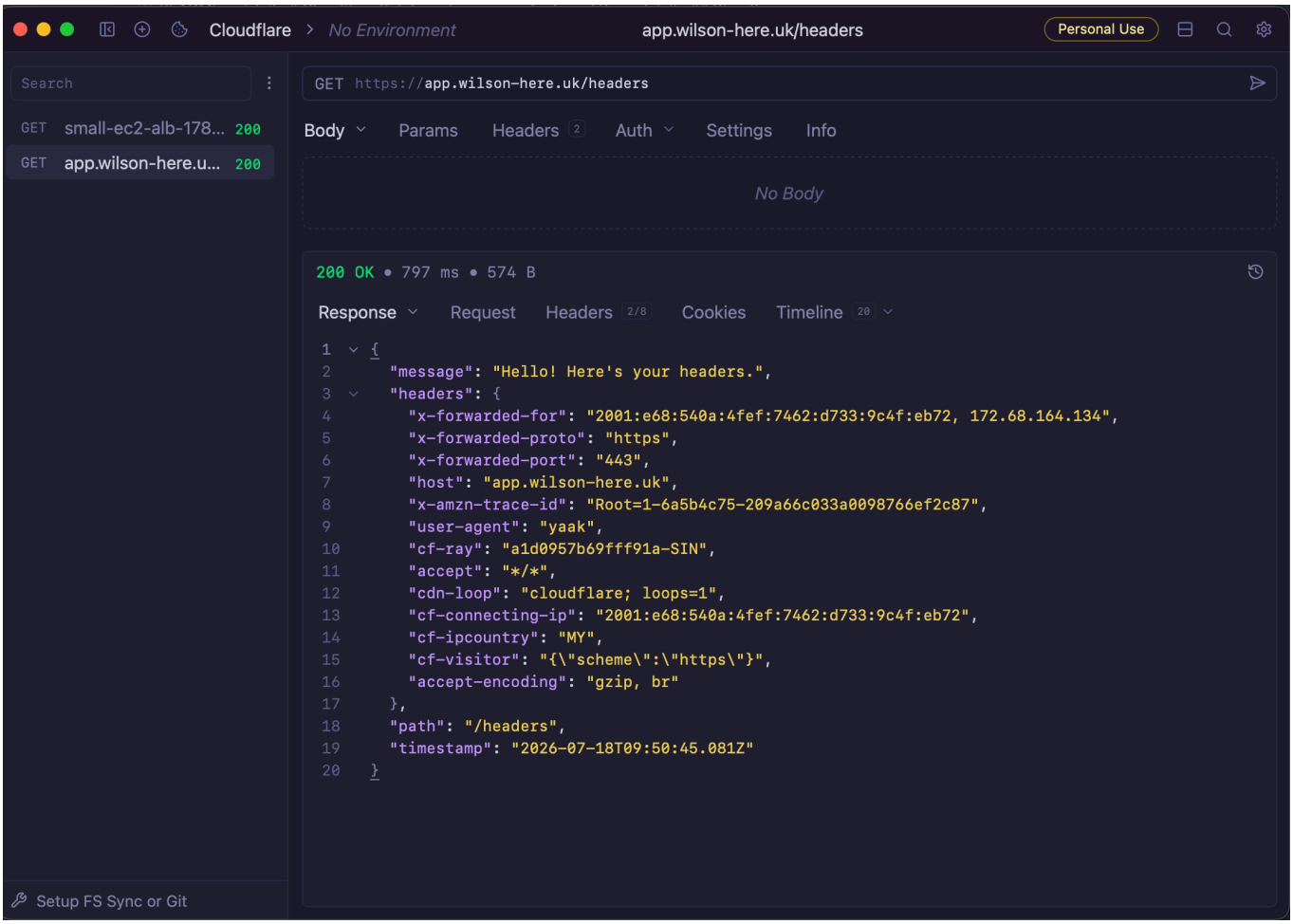
After changing SSL/TLS encryption on Cloudflare to Full(Strict), the https request was hit with `Error 526: invalid SSL certificate`.

And this issue

```
curl -v https://app.wilson-here.uk/headers

* Could not resolve host: app.wilson-here.uk
* Closing connection
curl: (6) Could not resolve host: app.wilson-here.uk
```

Resolved by flushing local DNS cache.

```
sudo dscacheutil -flushcache; sudo killall -HUP mDNSResponder
```


Cloudflare
No Environment
app.wilson-here.uk/headers
Personal Use
Search
GET small-ec2-alb-178... 200
GET app.wilson-here.u... 200
GET https://app.wilson-here.uk/headers
Body
Params
Headers 2
Auth
Settings
Info
No Body
200 OK • 797 ms • 574 B
Response
Request
Headers 2/8
Cookies
Timeline 20
{
"message": "Hello! Here's your headers.",
"headers": {
"x-forwarded-for": "2001:e68:540a:4fef:7462:d733:9c4f:eb72, 172.68.164.134",
"x-forwarded-proto": "https",
"x-forwarded-port": "443",
"host": "app.wilson-here.uk",
"x-amzn-trace-id": "Root=1-6a5b4c75-209a66c033a0098766ef2c87",
"user-agent": "yaak",
"cf-ray": "a1d0957b69fff91a-SIN",
"accept": "*/*",
"cdn-loop": "cloudflare; loops=1",
"cf-connecting-ip": "2001:e68:540a:4fef:7462:d733:9c4f:eb72",
"cf-ipcountry": "MY",
"cf-visitor": "{\"scheme\":\"https\"}",
"accept-encoding": "gzip, br"
},
"path": "/headers",
"timestamp": "2026-07-18T09:50:45.081Z"
}
Setup FS Sync or Git

dash.cloudflare.com/6320a1b4067829ac40a213cbf22df79d/wilson-here.uk/ssl-tls

wilson-here.uk free

Ask AI Support

Video guide Activate

SSL/TLS

Documentation Configure

Choose how Cloudflare encrypts traffic between your visitors and Cloudflare, and between Cloudflare and your origin server, to prevent data theft and other tampering.

SSL/TLS encryption

Current encryption mode: Full (strict)

Automatic mode disabled · 1 hour ago

Mode last changed · 1 minute ago

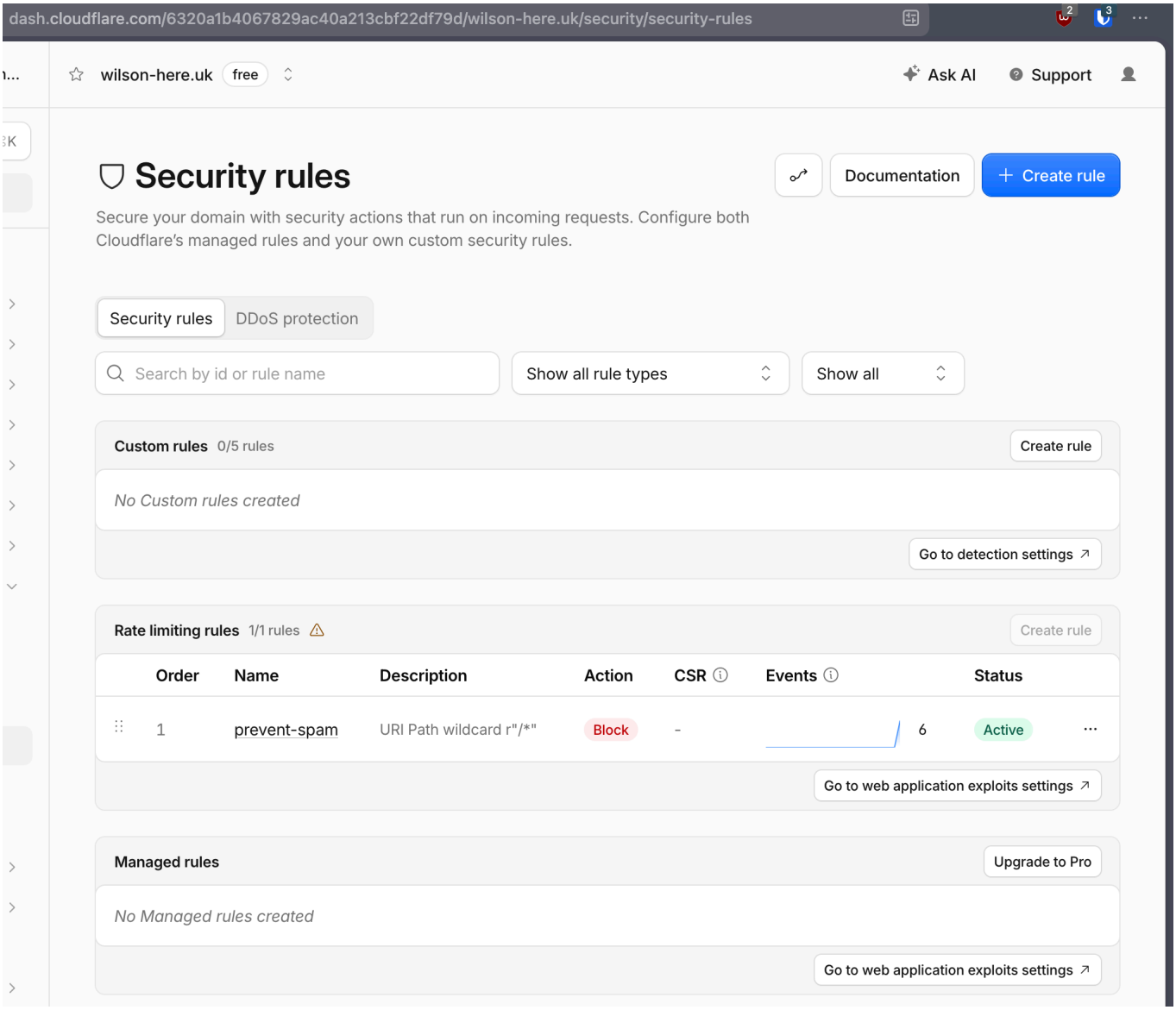
Traffic Served Over TLS

Last 24 hours

None (not secure)	TLS v1.3
20	13

Step 4

> Set a rate limiting rule on Cloudflare. How would you demonstrate to a customer that this rate limiting rule works?




```
hurl -v test_rate_limiting.hurl
* ------------------------------------------------------------------------
----
* Executing entry 1
*
* Entry options:
* delay: 1s
* repeat: 3
*
* Delay entry 1 (pause 1000 ms)
*
```

```
* Cookie store:
*
* Request:
* GET https://app.wilson-here.uk/headers
*
* Request can be run with the following curl command:
* curl 'https://app.wilson-here.uk/headers'
*
> GET /headers HTTP/2
> Host: app.wilson-here.uk
> Accept: */*
> User-Agent: hurl/8.0.1
>
* Response: (received 580 bytes in 312 ms)
*
< HTTP/2 200
< date: Sat, 18 Jul 2026 10:25:01 GMT
< content-type: application/json
< cf-cache-status: DYNAMIC
< nel: {"report_to":"cf-nel","success_fraction":0.0,"max_age":604800}
< report-to: {"group":"cf-nel","max_age":604800,"endpoints":
[{"url":"https://a.nel.cloudflare.com/report/v4?
s=fvtpjvEJyJOLx5CbFQ%2Bboth%2BtzKVO2u9%2Ft40lild4ISVI2BZLjNBYytaSO%2FrY%2B3U
3V9HW5HSNOQSh2zg5JEvJ%2FdWEP7%2FIDScQG77Fmgc3JIq3itMm8DHuGeBzIH652QlMi97SiKE
nR%2BUPz3cx4%2BYt5U%3D"}]}
< server: cloudflare
< cf-ray: a1d0c7aeec1ed4de-SIN
< alt-svc: h3=":443"; ma=86400
<
*
* Repeat entry 1 (x1/3)
* ------------------------------------------------------------------------
----
* Executing entry 1
*
* Entry options:
* delay: 1s
* repeat: 3
*
* Delay entry 1 (pause 1000 ms)
*
* Cookie store:
*
* Request:
* GET https://app.wilson-here.uk/headers
*
```

```
* Request can be run with the following curl command:
* curl 'https://app.wilson-here.uk/headers'
*
> GET /headers HTTP/2
> Host: app.wilson-here.uk
> Accept: */*
> User-Agent: hurl/8.0.1
>
* Response: (received 17 bytes in 69 ms)
*
< HTTP/2 429
< date: Sat, 18 Jul 2026 10:25:02 GMT
< content-type: text/plain; charset=UTF-8
< content-length: 17
< retry-after: 10
< cache-control: private, max-age=0, no-store, no-cache, must-revalidate,
post-check=0, pre-check=0
< expires: Thu, 01 Jan 1970 00:00:01 GMT
< referrer-policy: same-origin
< x-frame-options: SAMEORIGIN
< report-to: {"group":"cf-nel","max_age":604800,"endpoints":
[{"url":"https://a.nel.cloudflare.com/report/v4?
s=JMekU5PKHk8gYp03vhODGj8gOcXHDVvGKVsBEGWfxgxu2%2BpKvA1PNBuDl5AgeTokcdJO01I9
2ztL%2BuFgl5HCvI%2BzUEfKf8yOx%2F5bfUoFSKpV%2Fd%2BMjyyo1GxH4oYspGGeg%2Bp04z2D
1Xu3uc7RIUjV5Ds%3D"}]}
< nel: {"report_to":"cf-nel","success_fraction":0.0,"max_age":604800}
< server: cloudflare
< cf-ray: a1d0c7b60b48d4de-SIN
< alt-svc: h3=":443"; ma=86400
<
*
* Repeat entry 1 (x2/3)
* ------------------------------------------------------------------------
----
* Executing entry 1
*
* Entry options:
* delay: 1s
* repeat: 3
*
* Delay entry 1 (pause 1000 ms)
*
* Cookie store:
*
* Request:
* GET https://app.wilson-here.uk/headers
```

```
*
* Request can be run with the following curl command:
* curl 'https://app.wilson-here.uk/headers'
*
> GET /headers HTTP/2
> Host: app.wilson-here.uk
> Accept: */*
> User-Agent: hurl/8.0.1
>
* Response: (received 17 bytes in 57 ms)
*
< HTTP/2 429
< date: Sat, 18 Jul 2026 10:25:03 GMT
< content-type: text/plain; charset=UTF-8
< content-length: 17
< retry-after: 9
< cache-control: private, max-age=0, no-store, no-cache, must-revalidate,
post-check=0, pre-check=0
< expires: Thu, 01 Jan 1970 00:00:01 GMT
< referrer-policy: same-origin
< x-frame-options: SAMEORIGIN
< report-to: {"group":"cf-nel","max_age":604800,"endpoints":
[{"url":"https://a.nel.cloudflare.com/report/v4?
s=kRIl1nlfhU61smEO2K5csNtzf5ZEdcE3mRUcGfi4Oo7ZmsM7G2mmYrNoekdNsPmjo5TqtvxDeL
cdJ2Xf5pKR%2FPtb%2BkIgC1HvVDUSjY4vy3mJbg72yiwWnMlVjXjh3jLPHyXlLu%2B%2Fhsyp7O
p6fqB3E%2Bg%3D"}]}
< nel: {"report_to":"cf-nel","success_fraction":0.0,"max_age":604800}
< server: cloudflare
< cf-ray: a1d0c7bcb97fd4de-SIN
< alt-svc: h3=":443"; ma=86400
<
*
error code: 1015
```

Step 5

> Install and configure Cloudflare Tunnel on your origin server using a subdomain called “tunnel”, e.g. tunnel.yourwebsite.com. Make connections proxied to your server protected using this tunnel.

Followed instructions from this page:
https://developers.cloudflare.com/tunnel/deployment-guides/aws/#2-create-a-tunnel

To secure the AWS EC2, for the Security Group rules, only allow inbound traffic for port 22 (SSH) and TCP from AWS ALB and allow only outbound traffic to the Cloudflare Tunnel IP addresses. All Security Group rules are Allow rules; traffic that does not match a rule is blocked. Therefore, you can delete all inbound rules and leave only the relevant outbound rules.

Step 6

> Configure an SSO IdP (Identity Provider) of your choosing within Cloudflare Zero Trust. Please note that Cloudflare also provides an “OTP” option, in case you do not have any IdPs available to configure.

Step 7

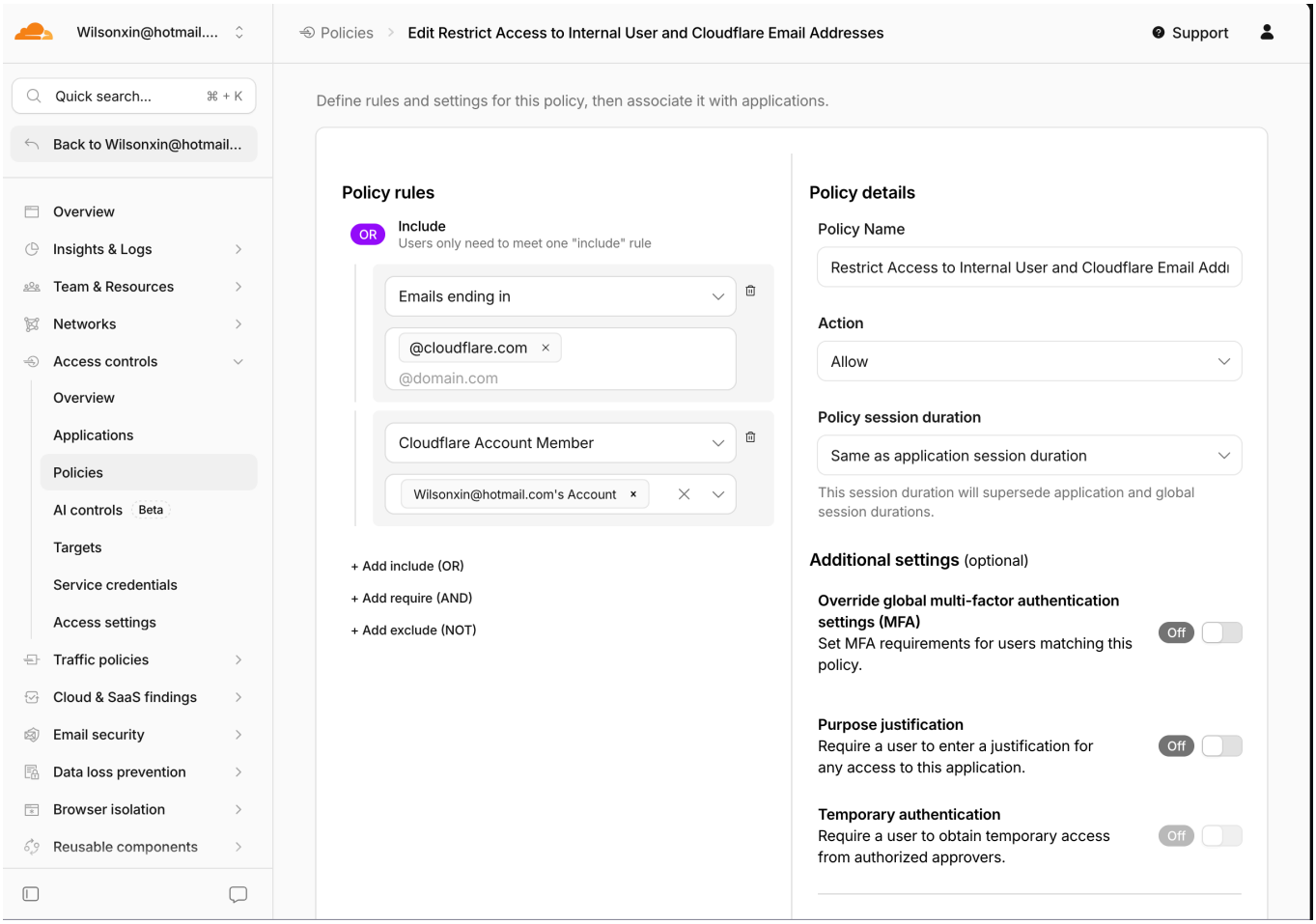
> Lock down access for a particular path for your Cloudflare Tunnel subdomain (e.g. tunnel.yourwebsite.com/secure) and only allow access for yourself (with the previously configured IdP of your choosing) and users with an @cloudflare.com email address .
> a. Ensure nobody can bypass Cloudflare and access your server’s IP directly.

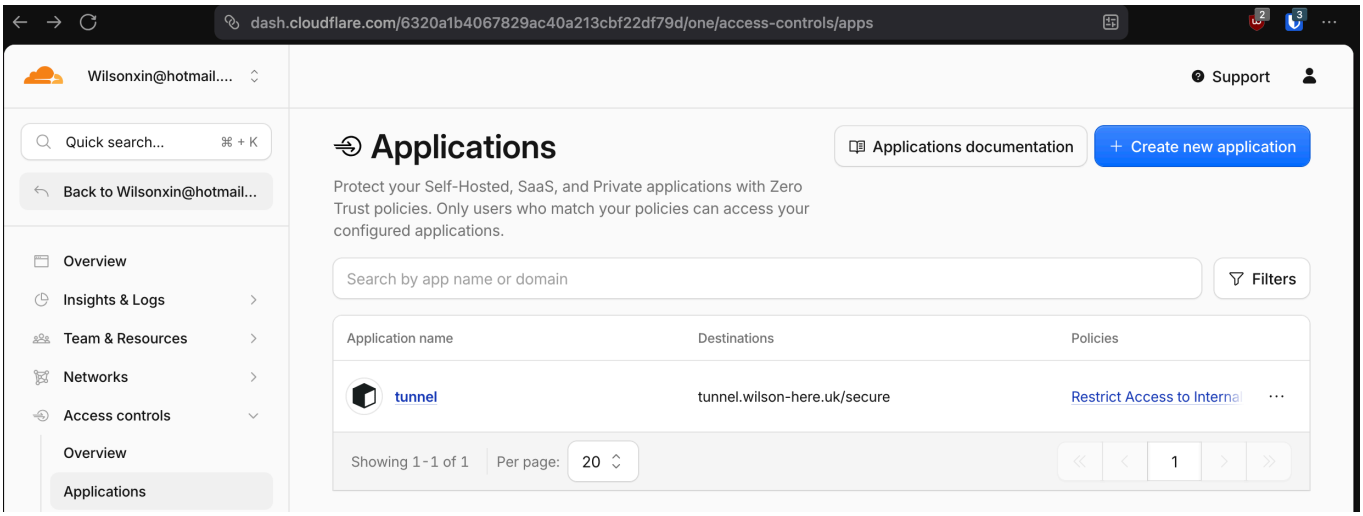
- https://developers.cloudflare.com/cloudflare-one/integrations/identity-providers/one-time-pin/

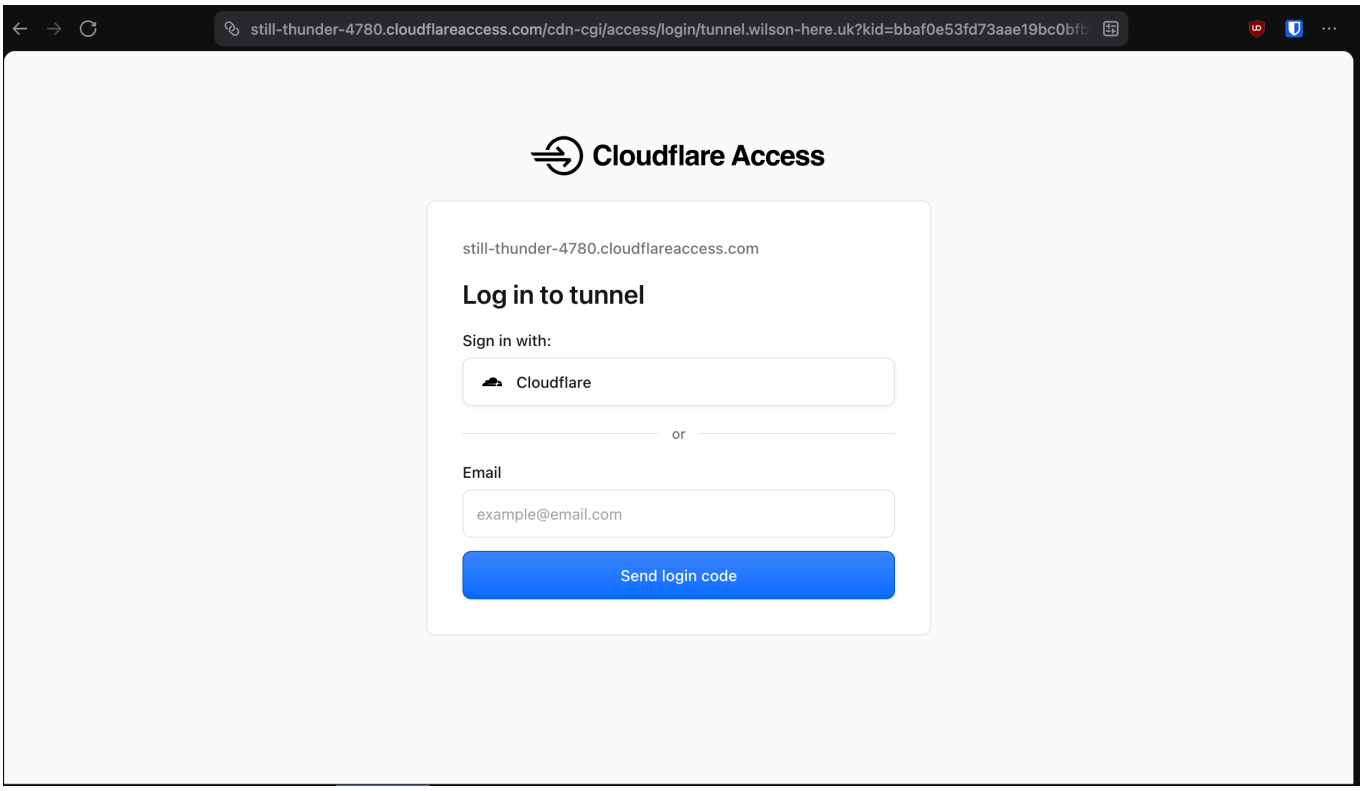
Configured Cloudflare OTP and IdP and Secured Path

Path: tunnel.wilson-here.uk/secure

(which includes tunnel.wilson-here.uk/secure/{country})


Wilsonxin@hotmail....
Policies
Edit Restrict Access to Internal User and Cloudflare Email Addresses
Support
Quick search... ⌘ + K
Back to Wilsonxin@hotmail...
Overview
Insights & Logs
Team & Resources
Networks
Access controls
Overview
Applications
Policies
AI controls Beta
Targets
Service credentials
Access settings
Traffic policies
Cloud & SaaS findings
Email security
Data loss prevention
Browser isolation
Reusable components
Define rules and settings for this policy, then associate it with applications.
Policy rules
OR
Include
Users only need to meet one "include" rule
Emails ending in
@cloudflare.com
@domain.com
Cloudflare Account Member
Wilsonxin@hotmail.com's Account
+ Add include (OR)
+ Add require (AND)
+ Add exclude (NOT)
Policy details
Policy Name
Restrict Access to Internal User and Cloudflare Email Addı
Action
Allow
Policy session duration
Same as application session duration
This session duration will supersede application and global session durations.
Additional settings (optional)
Override global multi-factor authentication settings (MFA)
Set MFA requirements for users matching this policy.
Off
Purpose justification
Require a user to enter a justification for any access to this application.
Off
Temporary authentication
Require a user to obtain temporary access from authorized approvers.
Off


dash.cloudflare.com/6320a1b4067829ac40a213cbf22df79d/one/access-controls/apps
Wilsonxin@hotmail....
Support
Quick search... ⌘ + K
Back to Wilsonxin@hotmail...
Overview
Insights & Logs
Team & Resources
Networks
Access controls
Overview
Applications
Applications
Protect your Self-Hosted, SaaS, and Private applications with Zero Trust policies. Only users who match your policies can access your configured applications.
Applications documentation
Create new application
Search by app name or domain
Filters
Application name
Destinations
Policies
tunnel
tunnel.wilson-here.uk/secure
Restrict Access to Interna
Showing 1-1 of 1
Per page: 20
1

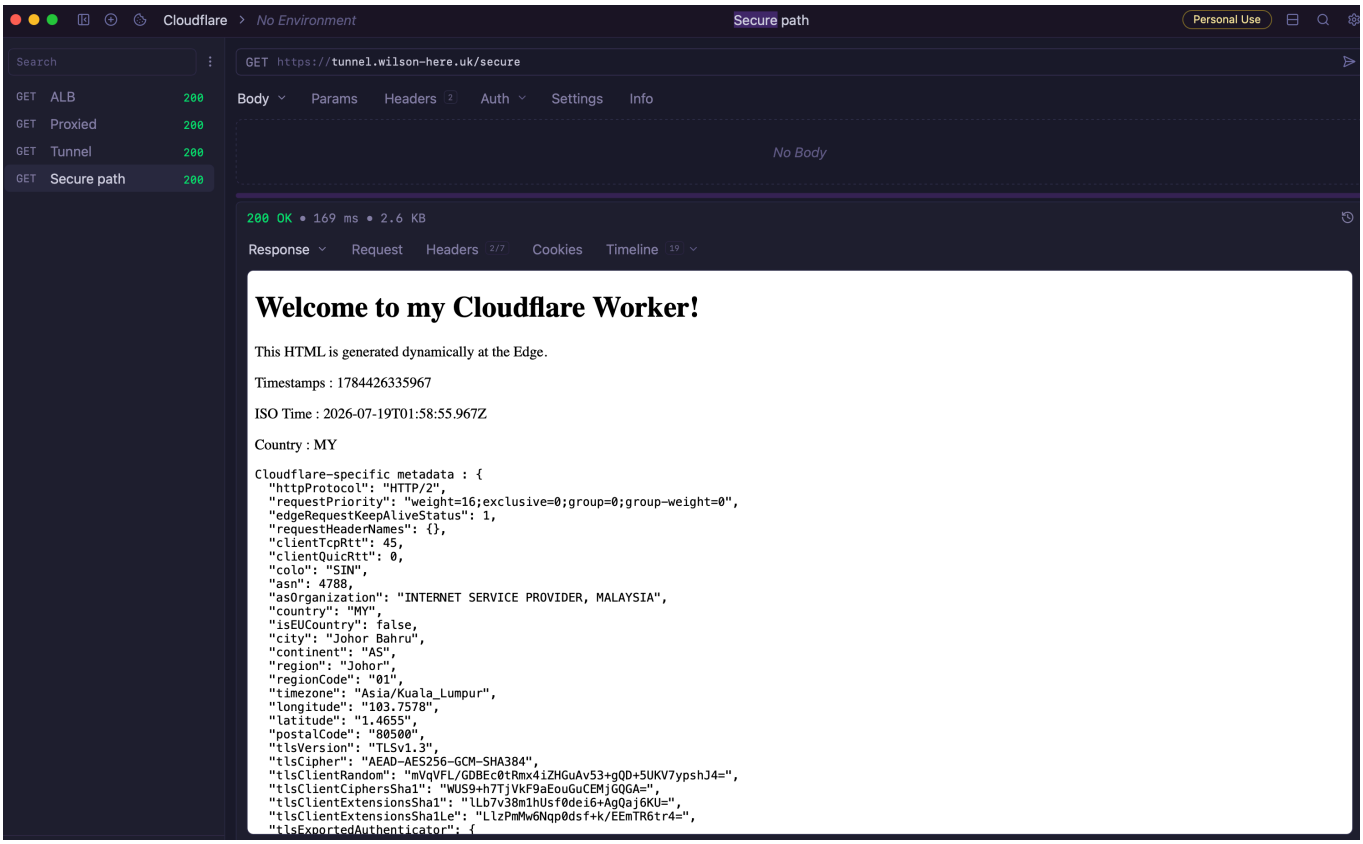

still-thunder-4780.cloudflareaccess.com/cdn-cgi/access/login/tunnel.wilson-here.uk?kid=bbaf0e53fd73aae19bc0bfb
Cloudflare Access
still-thunder-4780.cloudflareaccess.com
Log in to tunnel
Sign in with:
Cloudflare
or
Email
example@email.com
Send login code

Step 8

Setup a worker returning HTML with partial required info

Followed instructions here:

- https://developers.cloudflare.com/workers/wrangler/commands/workers/
- https://developers.cloudflare.com/workers/wrangler/configuration/
- https://developers.cloudflare.com/workers/configuration/routing/routes/
- https://claytonerrington.com/blog/using-cloud-flare-workers-to-get-visitor-information/



Set up R2 bucket, upload flags, display svg

- https://developers.cloudflare.com/workers/tutorials/upload-assets-with-r2/
- https://flagicons.lipis.dev/

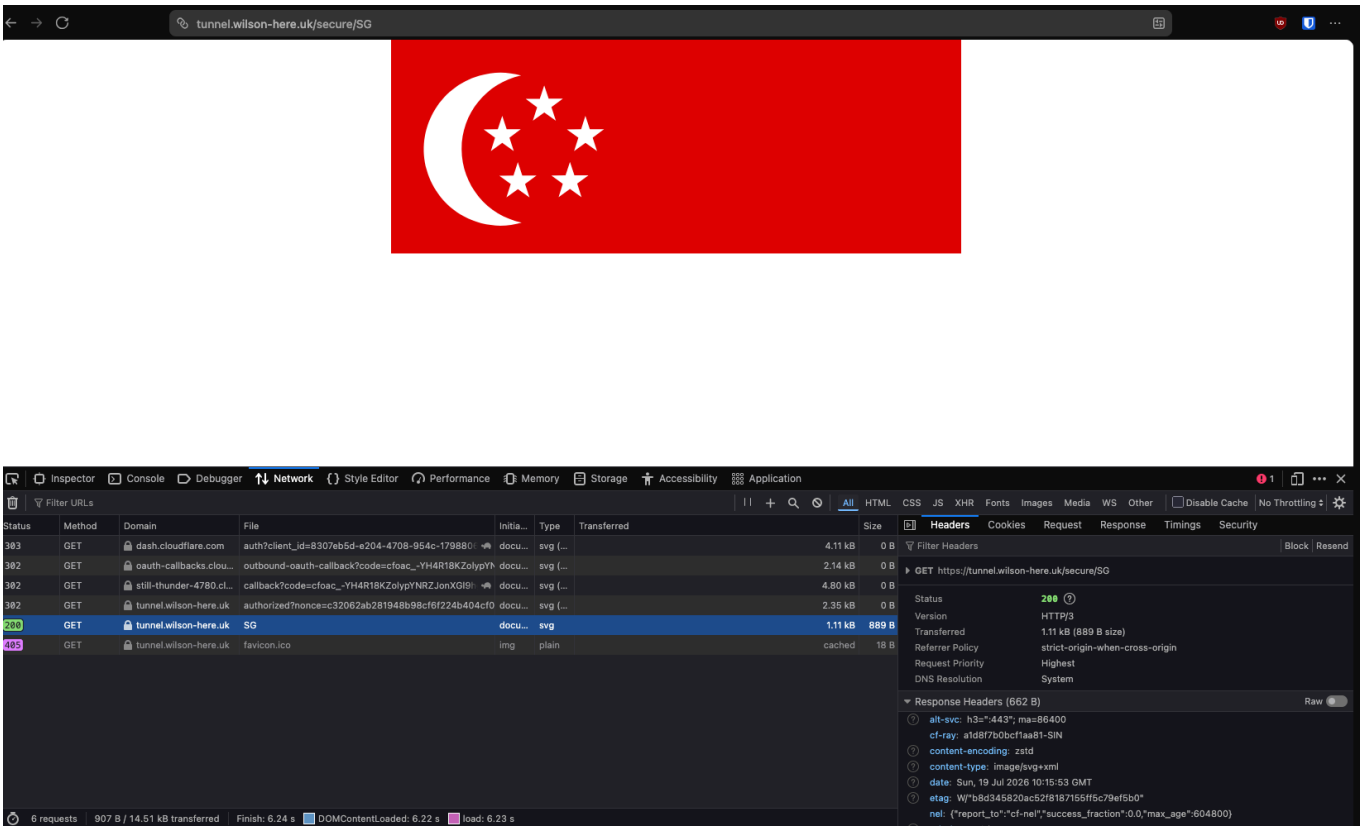
```
# Needs to enable R2 on the Cloudflare Dashboard
npx wrangler r2 bucket create country-flag-bucket
c
npx wrangler r2 bucket list

npx wrangler r2 object put country-flag-bucket/my.svg --
file=../assets/flags-4x3/my.svg --remote
```

tunnel.wilson-here.uk/secure/

Welcome to my Cloudflare Worker!

wilsonxin@hotmail.com authenticated at 1784456029721 (2026-07-19T10:13:49.721Z) from SG


```
Cloudflare-specific metadata : {
  "httpProtocol": "HTTP/3",
  "clientAcceptEncoding": "gzip, deflate, br",
  "requestPriority": "",
  "edgeRequestKeepAliveStatus": 1,
  "requestHeaderNames": {},
  "clientTcpRtt": 0,
  "clientQuicRtt": 7,
  "colo": "SIN",
  "asn": 4817,
  "asOrganization": "Simba Telecom Pte Ltd",
  "country": "SG",
  "isEUCountry": false,
  "city": "Woodlands",
  "continent": "AS",
  "timezone": "Asia/Singapore",
  "longitude": "103.78877",
  "latitude": "1.43801",
  "postalCode": "730101",
  "tlsVersion": "TLSv1.3",
  "tlsCipher": "AEAD-AES128-GCM-SHA256",
  "tlsClientRandom": "7bDnz+B73q3KcESrz1T7ArgGdir4eMF0ejUghawOvlw=",
  "tlsClientCiphersSha1": "majgAKUWrKK75XCpDQDr/Sf7Eyw=",
  "tlsClientExtensionsSha1": "6w9dpisjqFahizNHHZsPV4cLsZo=",
  "tlsClientExtensionsSha1Le": "Yc8UsW1DgZClRVm9Ry36tZSRNlc=",
  "tlsClientHelloLength": "1826",
```

Notes

To test EC2 public connection.

Then try each of these against it:

```
curl -m 5 http://<public-ip>/          # expect: timeout/connection refused
curl -m 5 http://<public-ip>:3000/     # expect: timeout/connection refused
nc -zv -w 3 <public-ip> 22             # expect: succeeds (SSH still open)
nc -zv -w 3 <public-ip> 80             # expect: fails
nc -zv -w 3 <public-ip> 3000           # expect: fails
nmap -Pn <public-ip>                   # expect: only 22 shows
open/filtered,  rest closed
```

Command	What it does	Why
`terraform output instance_public_ip`	Reads the EC2 instance's public IP from Terraform state	Need the actual IP before testing anything against it
`curl -m 5 http://<ip>/`	Sends an HTTP request to port 80, times out after 5s	Confirms the app isn't reachable on plain HTTP directly against the instance
`curl -m 5 http://<ip>:3000/`	Same, but against port 3000 (the app's actual port)	Confirms the app port isn't reachable directly either, only via the ALB
`nc -zv -w 3 <ip> 22`	`nc -z` = "just check if the port accepts a connection, don't send data"; `-v` verbose; `-w 3` = 3s timeout	Quick pass/fail port check, no HTTP involved — used here to confirm SSH *does* respond
`nc -zv -w 3 <ip> 80` / `nc -zv -w 3 <ip> 3000`	Same technique, against the ports that should be blocked	Confirms those ports refuse/timeout at the TCP level, not just at the HTTP layer
`nmap -Pn <ip>`	Scans all common ports on the host in one pass; `-Pn` skips the initial ping check (many EC2 instances don't respond to ICMP, so a scan without `-Pn` would falsely report the host as down)	One command to see the full picture — which ports are actually open vs. filtered/closed, instead of testing port-by-port